Colas

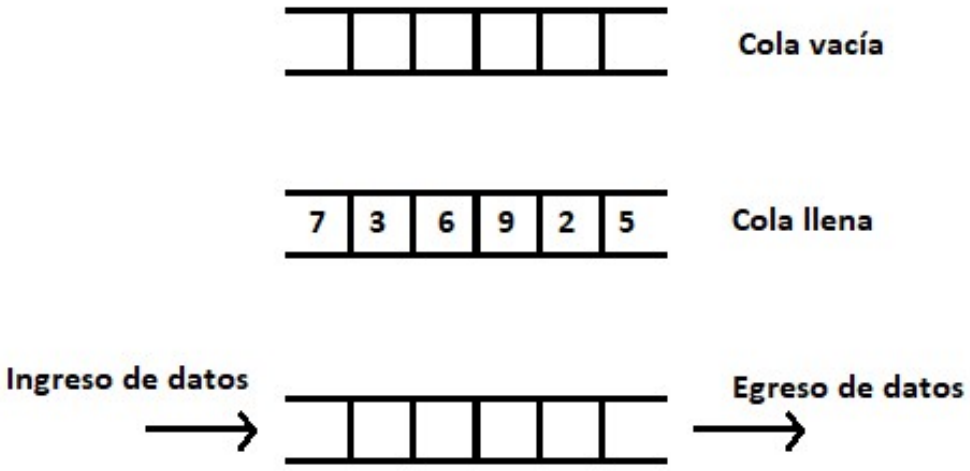
Esta estructura es similar a pila, trabaja como un contenedor de datos, pero con la diferencia que los datos ingresan por un extremo y salen por el otro. De tal manera que el primer dato que ingresó en la cola va a ser el primero en salir. Estas estructuras también se conocen como filas y responden al principio lógico FIFO (First In, First Out), lo que significa que el primer elemento en entrar es el primero en salir.

Para generalizar el concepto de la cola/fila pensemos en las colas que se realizan en un cajero, el primero en llegar va a ser el primero en ser atendido. La cola es una estructura de datos fundamental cuando se requiere procesar elementos en el orden en que llegaron, asegurando un flujo de procesamiento secuencial.

Aplicaciones de la Cola:

- Manejo de tareas en sistemas operativos (como la planificación de procesos).
- Simulación de sistemas de espera (por ejemplo, colas en bancos o supermercados).
- Gestión de recursos compartidos en redes y sistemas de impresión.
- Implementación de algoritmos de recorrido por niveles, como el BFS (Breadth-First Search) en grafos o árboles.

Esquema visual



La primera imagen la podemos representar como un vector vacío, al igual que en la pila este vector va a funcionar como contenedor de datos. La imagen del medio nos representa una cola llena y la última imagen nos muestra la secuencia de ingreso de datos.

Operaciones de una cola

- Enqueue/Agregar: Inserta un nuevo elemento en la cola, al final.
- Dequeue/Quitar: Remueve y devuelve el elemento que está en el frente.
- IsFully/Cola llena: Verifica si la cola está llena.
- IsEmpty/Cola vacía: Verifica si la cola está vacía.
- Inicializar: Establece valores iniciales para manipular la cola

Codificando una cola

```
#define poolSize 5

typedef struct cola
{
    int elementos[poolSize];
    int cantidad;
    int ultimo;
    int primero;
}tcola;
```

A fines prácticos vamos a implementar una cola que trabaja con enteros, pero debemos tener en cuenta que esta estructura puede ser más compleja y manipular otros tipos de datos como direcciones de memorias o sub estructuras internas.

La cola la armamos como una estructura con un vector del tamaño definido en la constante poolSize. Adicional a esto agregamos tres índices, el primero es el índice de cantidad que nos indica cuantos elementos hay cargados en el vector, índice que también utilizaremos para verificar el estado de la cola; el índice último nos indicará en qué posición del vector se encuentra el último dato ingresado; por otro lado, el índice primero indica cual es el primer dato ingresado por la tanto va a ser el primero en salir.

```
void inicializarCola(tcola *c)
{
    c->cantidad=0;
    c->primero=0;
    c->ultimo=0;
}
```

La función inicializar nos permite establecer valores por defecto a los índices de tal manera que cuando empecemos a ingresar datos, por ejemplo, el primer valor ocupe la posición 0 del vector.

```
int colaLlena(tcola *c)
{
    if(c->cantidad==poolSize)
        return 1;
    else
        return 0;
}
```

```
int colaVacia(tcola *c)
{
    if(c->cantidad==0)
        return 1;
    else
        return 0;
}
```

Ambas funciones de cola llena y cola vacía van verificando el estado del índice cantidad, y ambas quedan trabajando como funciones booleanas.

Agregando datos

Antes de agregar un dato, debemos verificar que existe lugar disponible en la pila, para eso recurrimos a la función pila llena. Una vez agregado el dato procedemos a modificar los índices correspondientes.

```
void agregarCola(tcola *c,int dato)
{
    if(!colaLlena(c))
    {
        c->elementos[c->ultimo]=dato;
        c->cantidad++;
        c->ultimo++;
        if(c->ultimo==poolSize)
            c->ultimo=0;
    }
    else{
        printf("Cola llena\n");
    }
}
```

Quitando datos

Al contrario de la función agregar, para quitar un dato es necesario verificar primero que existan datos en la estructura. En vez verificado esto podemos guardar el dato a quitar, modificar los índices correspondientes y luego devolver el dato.

```
int quitarCola(tcola*c)
{
    if(!colaVacia(c))
    {
        int dato=c->elementos[c->primero];
        c->cantidad--;
        c->primero++;
        if(c->primero==poolSize)
            c->primero=0;
        return dato;
    }else{
        printf("Cola vacia\n");
    }
}
```

A partir de este punto con estas funciones básicas podemos empezar a trabajar con la cola sin acceder a los elementos internos de la misma. Veamos un caso en donde queremos mostrar todos los elementos que contiene una cola. Si partimos del principio lógico que no podemos acceder al vector, debemos pensar en utilizar las funciones creadas para manipular la cola. Entonces deberíamos ir quitando los datos mientras la cola no esté vacía, mostrar los datos, guardar estos datos para no perderlos y luego devolverlos a la pila original para que no quede alterada.

Recorremos y mostramos los datos de una pila

```
void recorreCola(tcola *c)
{
    tcola auxcola;
    inicializarCola(&auxcola);
    while(!colaVacia(c))
    {
        int dato=quitarCola(c);
        printf("%d\n",dato);
        agregarCola(&auxcola,dato);
    }
    while(!colaVacia(&auxcola))
    {
        agregarCola(c,quitarCola(&auxcola));
    }
}
```

Aplicando la estructura en un programa:

```
int main()
{
    tcola nuevacola;
    inicializarCola(&nuevacola);
    agregarCola(&nuevacola,7);
    agregarCola(&nuevacola,9);
    agregarCola(&nuevacola,3);
    agregarCola(&nuevacola,1);
    agregarCola(&nuevacola,6);
    printf("Mostrando los valores de la cola\n");
    recorreCola(&nuevacola);
    quitarCola(&nuevacola);
    printf("Quitamos un valor\n");
    printf("Volvemos a mostrar\n");
    recorreCola(&nuevacola);

}
```